



TUBAF

The University of Resources.
Since 1765.



Practical Notebook Guide

OpenGeoSys Training Course

Running the Tunnel Excavation Example on Binder

Mehran Ghasabeh

October 2026

Institute of Geotechnics, TU Bergakademie Freiberg

Joint OpenGeoSys Training Course – UFZ, KIGAM, Zhejiang University, and TUBAF

Contents

1	Launch and Run	3
1.1	Launch link	3
1.2	Check OGS	3
1.3	Save results	4
2	Model and Binder Configuration	5
2.1	Tunnel example	5
2.2	Configuration files	5
2.3	Publish changes	6
3	Troubleshooting	7
3.1	OGSTools cannot be installed	7
3.2	Gmsh reports a missing library	7
3.3	Project files were not found	7
3.4	A build or run appears stuck	7
3.5	Avoid repeated GitHub authentication	8
3.6	References	8

1 Launch and Run

Binder starts a temporary online Jupyter environment from the GitHub repository. You can generate the meshes, run OpenGeoSys, and view the Kirsch benchmark plots in a browser without installing OGS locally. An internet connection is required.

1.1 Launch link

Bookmark this permanent link rather than the temporary Jupyter session address:

[Launch the OGS tunnel notebook on Binder](#)

1. Open the link. Wait for the image build and session startup; expand the build logs to follow progress.
2. When JupyterLab opens, confirm that the tunnel notebook is visible and the Python kernel is connected.
3. Choose **Run** → **Run All Cells**. Start with the imports and execute the cells in order.
4. Wait for mesh generation, project generation, the OGS simulations, and postprocessing. A busy kernel indicates ongoing execution.

The first launch after configuration changes can take several minutes. Later launches of the same commit usually reuse the image, but session startup still takes time. Service demand and cache availability affect the duration.

1.2 Check OGS

Open **File** → **New** → **Terminal** in Binder and run:

```
ogs --version
```

In a notebook cell, check Python and OGSTools with:

```
import sys
import shutil
import ogstools as ot
print(sys.version)
print(shutil.which("ogs"))
print(ot.__version__)
```

1.3 Save results

Simulation files are generated under:

```
01_Tunnel_Excavation_Kirsch/_out/
```

Download results and edited notebooks through JupyterLab before closing the session. Binder is temporary: a later launch does not restore the previous session's changes. To archive all results, run this from the repository root in Binder's terminal, then download the ZIP:

```
python -m zipfile -c tunnel_results.zip 01_Tunnel_Excavation_Kirsch/_out
```

2 Model and Binder Configuration

2.1 Tunnel example

The notebook models a quarter of a circular tunnel under plane-strain conditions in a linearly elastic medium. It compares release of nodal forces with a standard elastic approach and the analytical Kirsch solution, using linear and quadratic elements. An element-deactivation variant is also included.

Parameter	Value
Young's modulus	5 GPa
Poisson's ratio	0.30
Vertical compression magnitude	20 MPa
Tunnel radius	6.5 m
Tunnel centre	(0, -857) m
Domain extent from symmetry axes	70 m
Excavation duration	2 days
Simulation end time	4 days
Default linear solver	SparseLU

Gravity is disabled and the in-situ stress is prescribed. The tunnel centre only locates the mesh. The tension-positive initial stress is $(0, -20, -6, 0)$ MPa. Symmetry acts on the left and bottom boundaries, vertical traction on the top, and the right boundary is traction-free. Excavation forces on the tunnel arc decay linearly over two days.

This guide documents the workflow and repository inputs; it does not claim a newly verified numerical run.

2.2 Configuration files

All Binder configuration files below belong in the repository root.

runtime.txt

```
python-3.11
```

OGSTools 0.8 requires Python 3.11 or newer. This setting avoids the older Python environment used in the initial failed build.

Root requirements.txt

```
-r 01_Tunnel_Excavation_Kirsch/requirements.txt
gmsh
matplotlib
```

The example's included requirements specify:

```
ogstools[ogs]>=0.8,<0.9
jupyterlab>=4,<5
```

The OGS extra requests the executable wheel. OGSTools 0.8 provides the notebook's Model and Execution API. These version ranges do not freeze every dependency; future builds can resolve different versions.

apt.txt

```
libglul-mesa
libgl1
libgomp1
libxcursor1
libxinerama1
libxft2
libxext6
```

These supply native runtime and graphics libraries required by the Linux Gmsh wheel. Other required libraries are pulled in as package dependencies or provided by the base image.

postBuild

```
#!/bin/bash
set -eu
python -c "import gmsh; import ogstools; import pyvista; print('Notebook imports OK')
"
ogs --version
```

This executable script checks imports and OGS during image creation. It detects missing libraries before the notebook opens, but does not execute the full benchmark.

2.3 Publish changes

From the local repository directory, run:

```
git add requirements.txt apt.txt runtime.txt postBuild README.md
git commit -m "Configure Binder for the OGS tunnel example"
git push
```

Launch a new Binder session after pushing. An existing session does not receive newly added system packages.

3 Troubleshooting

3.1 OGSTools cannot be installed

For a build error mentioning Python 3.11 or newer and no matching OGSTools distribution, confirm that the root runtime file specifies Python 3.11. Push it and relaunch. Downgrading OGSTools would require adapting the notebook API.

3.2 Gmsh reports a missing library

The error `libXcursor.so.1: cannot open shared object file` indicates a missing native dependency, supplied by `libxcursor1`. Add the package to `apt.txt`, push, and launch a new session. Reinstalling the Python package alone does not install this library.

3.3 Project files were not found

If the kernel starts at the repository root rather than the example directory, restart it and run this new cell before the original import cell:

```
import os
from pathlib import Path
example = Path.cwd() / "01_Tunnel_Excavation_Kirsch"
if not example.is_dir():
    example = Path.cwd()
os.environ["KIRSCH_PROJECT_ROOT"] = str(example.resolve())
```

The selected directory must contain the mesh and project generator scripts.

3.4 A build or run appears stuck

Follow the Binder build log. If it fails, save the last 30–50 lines, including ERROR messages. In Jupyter, examine the first failing cell. Do not continue dependent cells after an import failure; resolve it and rerun from the beginning. Mesh generation and multiple simulations take time. Avoid submitting a cell repeatedly while the kernel is busy. If available resources are insufficient, use the local course environment.

3.5 Avoid repeated GitHub authentication

Participants only need the launch link. Git authentication is needed to push repository changes. On Ubuntu, authenticate once with GitHub CLI:

```
sudo apt update
sudo apt install gh
gh auth login --hostname github.com --git-protocol https --web
gh auth setup-git
```

Enter the one-time code displayed in your terminal on the GitHub browser page. The code is not sent by email. The credential setup applies to HTTPS repositories on github.com, subject to account permissions; SSH uses separate keys.

GitHub CLI uses a secure system credential store when available and otherwise falls back to a plain-text file. If Git asks directly for an HTTPS password, use a personal access token rather than your account password. Keep tokens and login codes private.

3.6 References

- [Binder/repo2docker configuration](#).
- [Python runtime and system packages](#).
- [OGSTools installation guide](#).
- [GitHub CLI authentication and storage](#).
- [GitHub CLI Git credentials](#).